Universidad Católica Boliviana

Maestría en Ciencia de Datos e Inteligencia Artificial

Asignatura: [MCI-511] Computación en la Nube



Portal académico en la nube

Reto 20 – Proyecto Final

Estudiante: Jhonny Paul Pinto Phillips
Correo: jhonny.pinto@ucb.edu.bo
Repositorio: github.com/paul-pinto/reto20-portal-academico
Reto: Reto 20 – Portal académico en la nube
Docente: Fabrizio Gustavo Bellido Parra
Fecha: 23 de octubre de 2026

Santa Cruz de la Sierra – Bolivia

Índice

1. Resumen ejecutivo **3**

2. Problema, objetivos y alcance **3**

2.1. Problema . . . 3

2.2. Objetivo general . . . 3

2.3. Objetivos específicos . . . 3

2.4. Alcance . . . 4

3. Arquitectura de la solución **4**

3.1. Arquitectura general . . . 4

3.2. Servicios AWS utilizados . . . 6

3.3. Componentes de software . . . 6

4. Implementación **6**

4.1. Secuencia de despliegue . . . 6

4.2. Capa de cómputo: Amazon EC2 . . . 7

4.3. Persistencia: Amazon DynamoDB . . . 7

4.4. Almacenamiento privado: Amazon S3 . . . 8

4.5. Autenticación y autorización . . . 8

4.6. Despliegue de la aplicación . . . 8

5. Pruebas y resultados **8**

5.1. Criterios de aceptación . . . 9

5.2. Prueba de estudiante . . . 9

5.3. Prueba de docente . . . 9

5.4. Acceso a archivos privados . . . 9

6. Seguridad, observabilidad y costos **10**

6.1. Seguridad . . . 10

6.2. Observabilidad y auditoría . . . 10

6.3. Costos y entorno académico 11

7. Limitaciones y ruta alternativa **11**

7.1. Servicios no habilitados y ruta alternativa 12

7.2. Dirección IPv4 pública dinámica 12

7.3. Uso de HTTP 12

7.4. Escalabilidad 12

8. Conclusiones y limpieza **12**

8.1. Limpieza de recursos 13

9. Referencias **13**

A. Evidencias de implementación **14**

1. Resumen ejecutivo

El proyecto responde a la necesidad de contar con un portal académico en el que estudiantes y docentes puedan trabajar con cursos, actividades y entregas sin exponer información ni archivos que no les correspondan. El problema central fue garantizar que cada usuario acceda únicamente a los recursos autorizados para su rol, manteniendo además un registro de los accesos relevantes.

Se construyó una solución funcional en la nube que centraliza los datos académicos, mantiene los archivos de entrega en almacenamiento privado y registra eventos de acceso y auditoría. La aplicación diferencia las funciones de estudiantes y docentes y valida los permisos en el servidor antes de mostrar información o entregar acceso a un archivo.

La prueba de aceptación fue satisfactoria: el estudiante pudo acceder a su propia entrega y fue bloqueado al intentar consultar una entrega ajena; del mismo modo, el docente accedió al curso que tenía asignado y recibió una respuesta HTTP 403 al intentar ingresar a un curso ajeno. En conjunto, las cuatro pruebas principales definidas para el proyecto se cumplieron correctamente.

2. Problema, objetivos y alcance

2.1. Problema

El caso de uso representa un portal académico real en el que intervienen dos grupos de usuarios: **estudiantes**, que consultan cursos y gestionan sus propias entregas, y **docentes**, que revisan actividades y entregas de los cursos que tienen asignados. El riesgo principal consiste en que un usuario pueda acceder a información o archivos pertenecientes a otra persona o a un curso que no le corresponde.

La solución debe, por tanto, controlar el acceso a información sensible y archivos privados, mantener trazabilidad de los accesos y operar sobre una infraestructura de nube. La prueba de aceptación definida exige que: (1) un estudiante pueda acceder a su propia entrega pero no a la de otro estudiante; y (2) un docente pueda revisar su propio curso pero no un curso ajeno.

2.2. Objetivo general

Diseñar e implementar un portal académico en AWS que permita gestionar usuarios, cursos, actividades, entregas y revisiones, incorporando controles de acceso, almacenamiento privado de archivos y mecanismos básicos de observabilidad.

2.3. Objetivos específicos

- Desplegar una aplicación web Flask en Amazon EC2.
- Implementar persistencia de datos mediante Amazon DynamoDB.

- Almacenar entregas académicas de forma privada en Amazon S3.
- Generar URLs prefirmadas para acceso temporal a archivos.
- Aplicar autorización diferenciada para estudiantes y docentes.
- Registrar eventos de auditoría en Amazon CloudWatch.
- Validar el cumplimiento de los criterios de aceptación definidos para el proyecto.

2.4. Alcance

El proyecto contempla un entorno académico demostrativo con datos sintéticos. Incluye autenticación, autorización por rol, gestión de cursos y actividades, acceso a entregas, revisión por parte de docentes, almacenamiento privado y auditoría de eventos relevantes.

No se contemplan funcionalidades de producción como alta disponibilidad multi-AZ, dominio personalizado, certificado TLS gestionado, balanceador de carga, recuperación ante desastres o integración con un proveedor de identidad externo.

3. Arquitectura de la solución

3.1. Arquitectura general

La arquitectura sigue un enfoque basado en servicios administrados de AWS y una aplicación web desplegada sobre una instancia EC2.

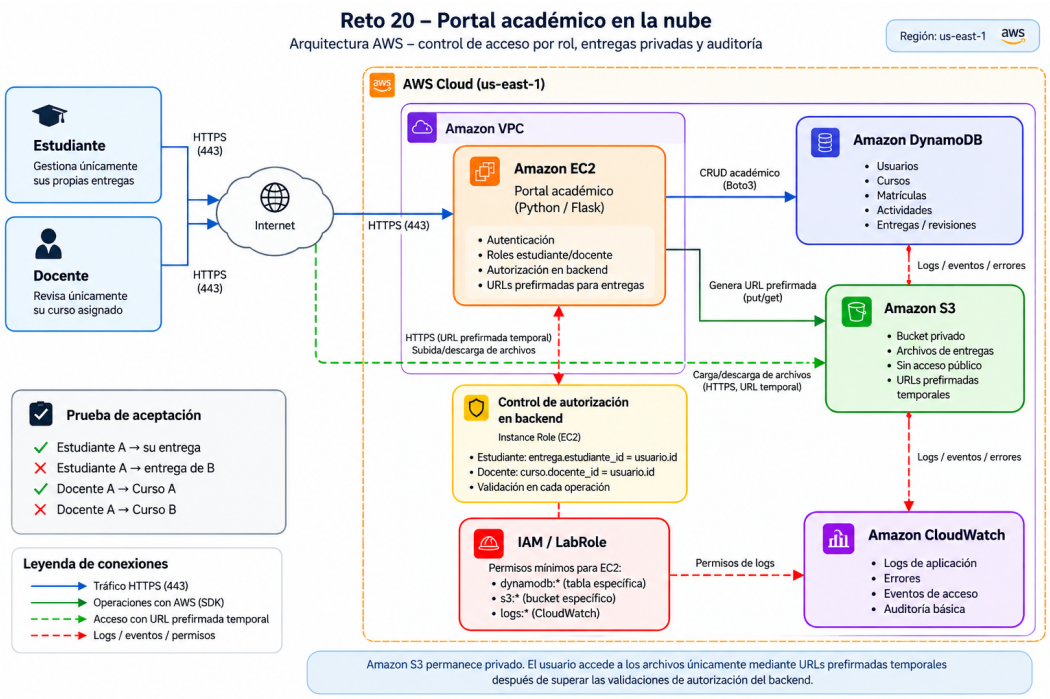


Figura 1: Arquitectura general del portal académico en AWS.

El flujo principal es el siguiente:

1. El usuario accede al portal mediante Internet.
2. El tráfico ingresa a la instancia EC2 a través de las reglas definidas en el Security Group.
3. Nginx recibe las solicitudes HTTP y las redirige a Gunicorn.
4. Gunicorn ejecuta la aplicación Flask.
5. Flask consulta DynamoDB para obtener usuarios, cursos, actividades, entregas y revisiones.
6. Cuando un usuario autorizado solicita un archivo privado, Flask genera una URL prefirmada de S3.
7. Los eventos relevantes de la aplicación se registran y envían a CloudWatch.
8. IAM autoriza a la instancia EC2 a consumir los servicios AWS necesarios mediante un Instance Profile.

3.2. Servicios AWS utilizados

Cuadro 1: Servicios AWS: elección y alternativa descartada

Servicio	Motivo de elección	Alternativa descartada
Amazon EC2	Permitió desplegar Flask, Gunicorn y Nginx con control completo del servidor.	Elastic Beanstalk, por restricciones de permisos del Learner Lab.
Amazon DynamoDB	Adecuado para un modelo académico simple, sin administrar un motor relacional y con facturación bajo demanda.	Amazon RDS, innecesario para el alcance y con mayor carga operativa.
Amazon S3	Diseñado para archivos privados, versionado y generación de URLs prefirmadas.	Almacenar archivos en EC2, menos durable y desacoplado.
Amazon CloudWatch	Centraliza logs y eventos de auditoría de la aplicación.	Logs únicamente locales en EC2, con menor visibilidad y trazabilidad.
AWS IAM	Permite que EC2 acceda a AWS mediante LabRole/LabInstanceProfile sin credenciales estáticas.	Claves de acceso guardadas en el servidor, opción descartada por seguridad.
Amazon VPC / Security Groups	Aíslan la instancia y restringen el tráfico por puerto y origen.	Exponer servicios sin controles de red específicos.

3.3. Componentes de software

Dentro de Amazon EC2 se utilizaron los siguientes componentes:

- **Nginx**: proxy inverso y punto de entrada HTTP.
- **Gunicorn**: servidor WSGI.
- **Flask**: framework de aplicación web.
- **Boto3**: SDK de AWS utilizado desde Python.
- **systemd**: administración de los servicios de aplicación.
- **Amazon CloudWatch Agent**: envío de logs del servidor a CloudWatch.

4. Implementación

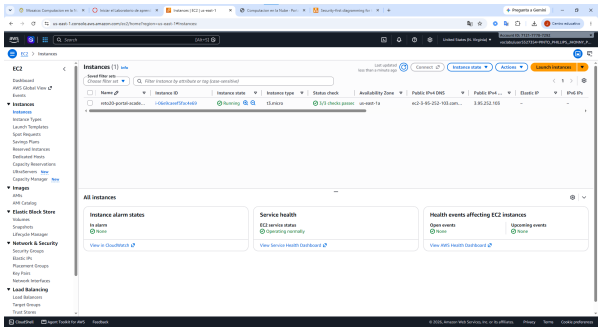
4.1. Secuencia de despliegue

La implementación se realizó en el siguiente orden: (1) creación y carga de datos en DynamoDB; (2) creación y endurecimiento del bucket S3; (3) despliegue de la instancia EC2 con LabInstanceProfile; (4) instalación de Flask, Gunicorn y Nginx; (5) configuración del servicio de systemd; (6) configuración del CloudWatch Agent; y (7) ejecución de las pruebas de aceptación desde el portal público.

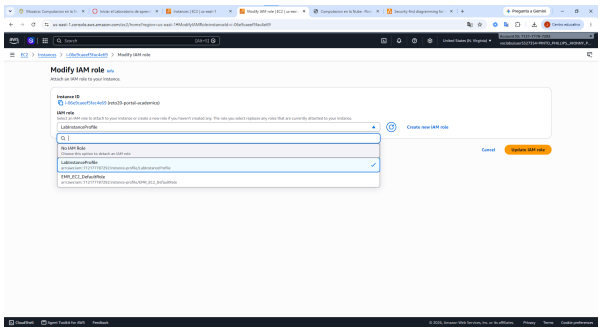
4.2. Capa de cómputo: Amazon EC2

La aplicación fue desplegada en una instancia Amazon Linux 2023. Se configuró un servicio de systemd para ejecutar Gunicorn de forma persistente y Nginx como proxy inverso.

La instancia utiliza un IAM Instance Profile para acceder a los servicios de AWS sin almacenar credenciales estáticas dentro del servidor.



(a) Instancia EC2 en ejecución.



(b) IAM Instance Profile asociado a la instancia.

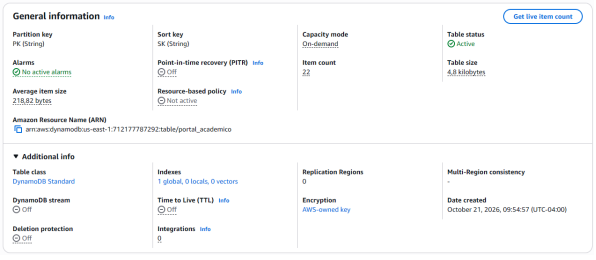
Figura 2: Capa de cómputo y autorización de la instancia EC2.

4.3. Persistencia: Amazon DynamoDB

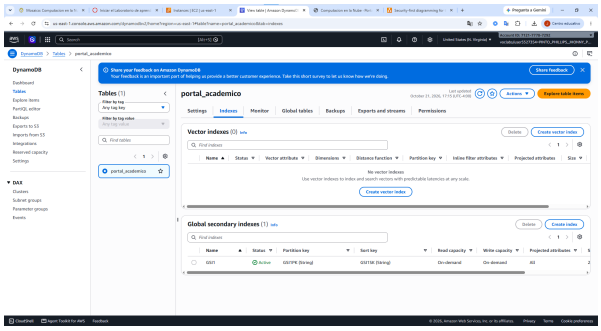
Se implementó una tabla denominada portal_academico, utilizando un diseño single-table con:

- clave de partición PK;
- clave de ordenamiento SK;
- índice secundario global GSI1;
- modo de facturación *PAY_PER_REQUEST*.

Las principales entidades almacenadas son USER, COURSE, ENROLLMENT, ACTIVITY, SUBMISSION y REVIEW.



(a) Claves PK y SK de la tabla.



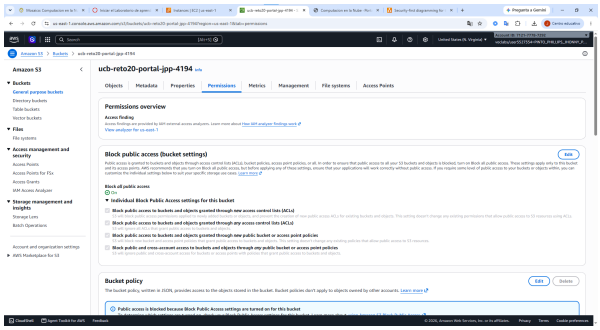
(b) Índice secundario global GSI1.

Figura 3: Diseño single-table implementado en Amazon DynamoDB.

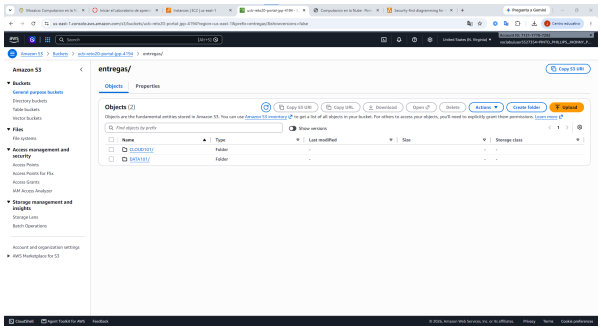
4.4. Almacenamiento privado: Amazon S3

Los archivos de entrega se almacenan en un bucket S3 privado. La configuración incluye bloqueo de acceso público, cifrado SSE-S3, versionado habilitado y objetos PDF con tipo de contenido application/pdf.

La aplicación no expone directamente los objetos de S3. Antes de generar una URL prefirmada, el backend valida que el usuario tenga autorización para acceder al archivo solicitado.



(a) Bloqueo de acceso público del bucket.



(b) Estructura de almacenamiento de las entregas.

Figura 4: Configuración de seguridad y organización del almacenamiento en Amazon S3.

4.5. Autenticación y autorización

El portal implementa autenticación mediante sesión y contraseñas almacenadas de forma segura utilizando PBKDF2-SHA256.

La autorización se valida en el backend con reglas específicas: un estudiante puede acceder únicamente a una entrega cuando el identificador del estudiante coincide con el usuario autenticado, mientras que un docente puede acceder únicamente a un curso cuando dicho curso tiene asignado a ese docente.

4.6. Despliegue de la aplicación

La aplicación se ejecuta mediante Gunicorn sobre el puerto local 8000 y Nginx expone el portal mediante el puerto HTTP 80.

$$\text{Internet} \rightarrow \text{Nginx} \rightarrow \text{Gunicorn} \rightarrow \text{Flask}$$

5. Pruebas y resultados

5.1. Criterios de aceptación

Cuadro 2: Resultados de las pruebas de aceptación

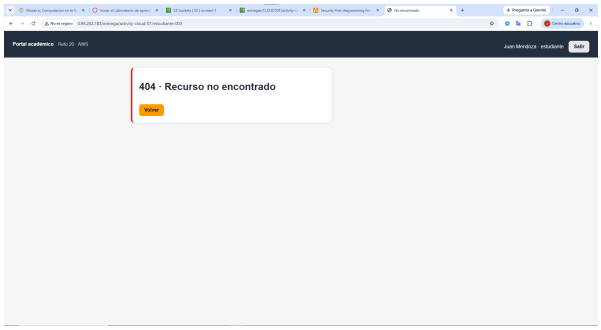
Prueba	Resultado esperado	Estado
Estudiante accede a su propia entrega	Acceso permitido	Cumple
Estudiante accede a entrega ajena	HTTP 403	Cumple
Docente accede a su propio curso	Acceso permitido	Cumple
Docente accede a curso ajeno	HTTP 403	Cumple

5.2. Prueba de estudiante

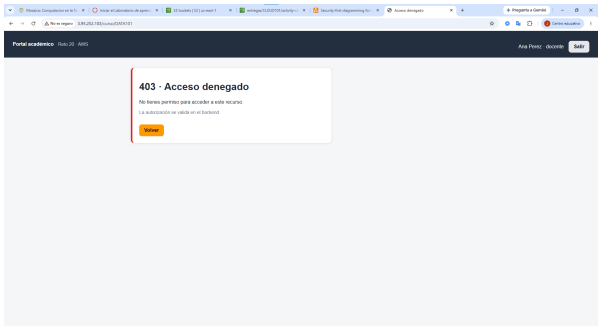
Se verificó que el usuario estudiante-001 pudiera acceder a su propia entrega y abrir el archivo privado mediante una URL prefirmada. También se intentó acceder a una entrega correspondiente a otro estudiante. El backend respondió con HTTP 403 y generó el evento de auditoría correspondiente.

5.3. Prueba de docente

Se verificó que el usuario profesor-001 pudiera acceder al curso CLOUD101, del cual es responsable. Al intentar acceder a DATA101, el sistema bloqueó la solicitud con HTTP 403.



(a) Estudiante: intento de acceso a una entrega ajena.



(b) Docente: intento de acceso a un curso ajeno.

Figura 5: Pruebas de autorización negativa con respuesta HTTP 403.

5.4. Acceso a archivos privados

Cuando un usuario autorizado solicita un archivo, la aplicación genera una URL prefirmada de duración limitada. De esta manera, el bucket permanece privado y el usuario no necesita permisos directos sobre S3.

Como resultado cuantitativo, se ejecutaron **4 pruebas principales de aceptación y las 4 cumplieron el resultado esperado (100 %)**. Los dos intentos de acceso no autorizado fueron rechazados mediante HTTP 403, mientras que los dos accesos autorizados se completaron correctamente.

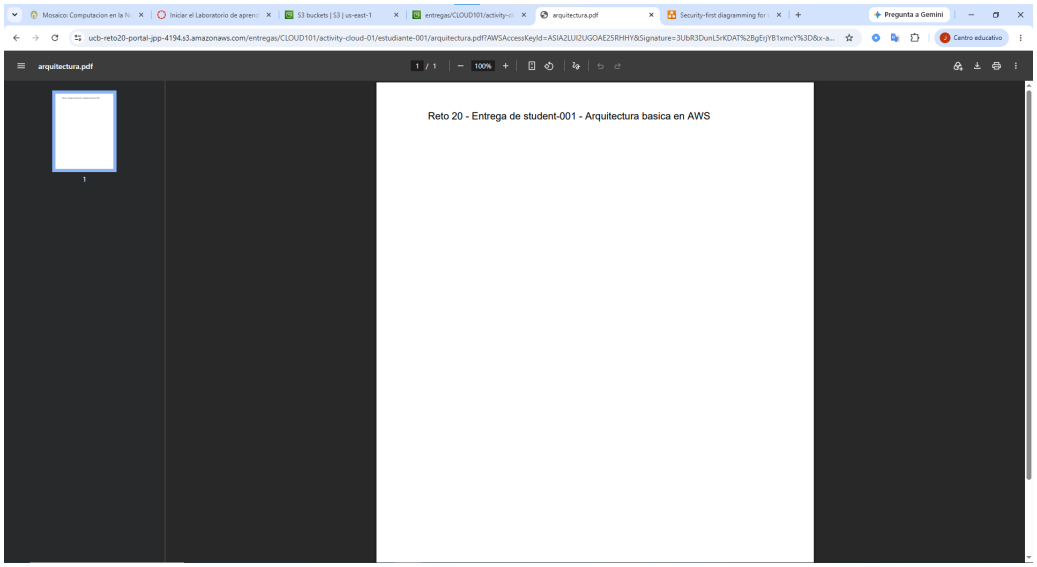


Figura 6: Acceso temporal a un archivo privado mediante una URL prefirmada de Amazon S3.

6. Seguridad, observabilidad y costos

6.1. Seguridad

Las principales medidas implementadas fueron:

- S3 con bloqueo total de acceso público y cifrado SSE-S3.
- Autorización aplicada en el backend y contraseñas almacenadas mediante hash PBKDF2-SHA256.
- Uso de **LabRole/LabInstanceProfile** para que EC2 consuma servicios AWS sin almacenar claves de acceso estáticas.
- La clave secreta de sesión de Flask se mantiene como variable de entorno del servicio y no se expone en el repositorio ni en el código fuente.
- SSH limitado a una dirección IP específica mediante regla /32.
- Exposición pública únicamente de los puertos necesarios.

6.2. Observabilidad y auditoría

La aplicación genera eventos de auditoría como login_success, login_failed, view_submission, presigned_url_generated, forbidden_course, forbidden_submission, forbidden_download y logout.

Los logs son enviados mediante CloudWatch Agent al grupo /reto20/portal-academico.

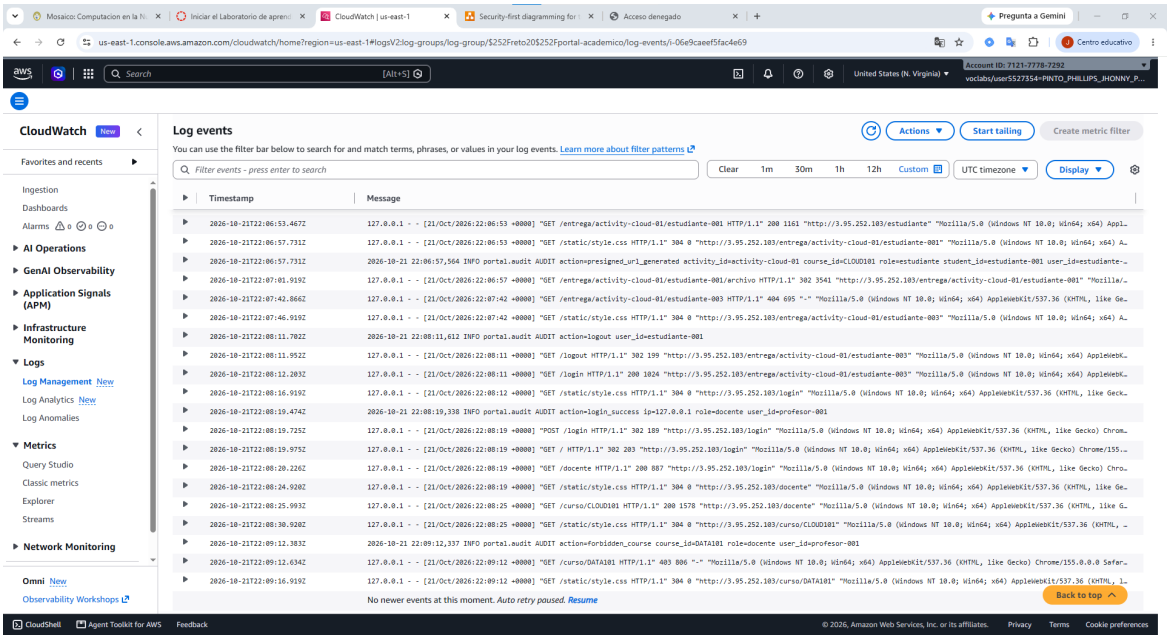


Figura 7: Flujo de logs y eventos de auditoría centralizados en Amazon CloudWatch.

6.3. Costos y entorno académico

La solución fue implementada dentro de AWS Academy Learner Lab. Se priorizaron servicios y configuraciones adecuadas para un entorno académico y de bajo costo, utilizando una instancia EC2 pequeña, DynamoDB bajo demanda y almacenamiento S3 reducido. El consumo fue controlado desde el panel del Learner Lab, cuya captura se conserva como evidencia.

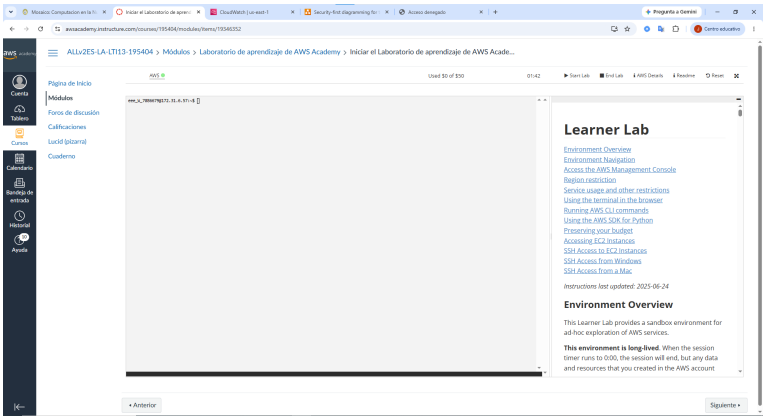


Figura 8: Consumo y créditos disponibles en AWS Academy Learner Lab.

7. Limitaciones y ruta alternativa

7.1. Servicios no habilitados y ruta alternativa

La ruta prevista contemplaba Elastic Beanstalk como opción administrada de despliegue. Debido a las restricciones de permisos asociadas al LabRole del AWS Academy Learner Lab, se utilizó **Amazon EC2 como ruta alternativa**. Esta decisión permitió completar el despliegue manteniendo control directo sobre Nginx, Gunicorn, Flask y el agente de CloudWatch.

7.2. Dirección IPv4 pública dinámica

La instancia no utiliza Elastic IP, por lo que la dirección IPv4 pública puede cambiar cuando la instancia es detenida y posteriormente iniciada.

7.3. Uso de HTTP

La demostración académica se implementó mediante HTTP. En un entorno de producción debería incorporarse HTTPS utilizando un certificado TLS válido.

7.4. Escalabilidad

La implementación actual utiliza una única instancia EC2 y no incorpora balanceo de carga ni autoescalado. Esto es suficiente para el alcance del reto, pero no representa una arquitectura de alta disponibilidad.

8. Conclusiones y limpieza

El proyecto permitió implementar satisfactoriamente un portal académico funcional en AWS integrando cómputo, persistencia, almacenamiento, seguridad y observabilidad. La principal lección aprendida fue que la seguridad no depende únicamente de ocultar opciones en la interfaz: las reglas deben validarse en el backend antes de entregar datos o archivos.

La separación entre datos estructurados en DynamoDB y archivos privados en S3 permitió utilizar cada servicio según su propósito. Las URLs prefirmadas mantuvieron los objetos privados y otorgaron acceso temporal únicamente después de validar la autorización.

Las cuatro pruebas principales de aceptación se cumplieron. Asimismo, CloudWatch permitió verificar accesos permitidos, intentos denegados y generación de URLs prefirmadas.

En una implementación de producción se incorporaría HTTPS, dominio personalizado, alta disponibilidad, balanceo de carga y un servicio especializado de identidad.

8.1. Limpieza de recursos

Los recursos implementados se mantienen disponibles hasta la realización de la defensa y demostración en AWS, debido a que forman parte de la prueba práctica del proyecto. Una vez concluida la exposición, los recursos que generen consumo serán detenidos o eliminados según corresponda. La evidencia final del estado de los recursos se actualizará inmediatamente después de la defensa, dejando constancia del cierre o detención de la infraestructura utilizada.

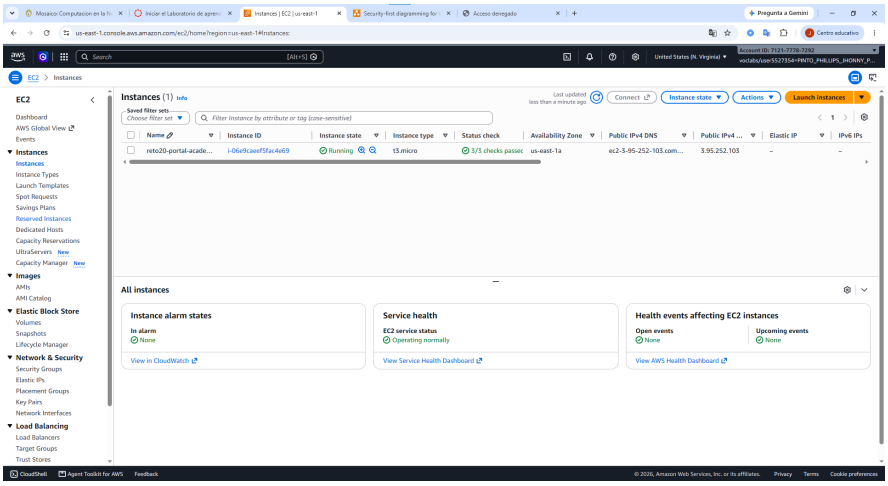


Figura 9: Estado final de los recursos después de la demostración.

9. Referencias

- Amazon Web Services. Documentación de Amazon EC2.
- Amazon Web Services. Documentación de Amazon DynamoDB.
- Amazon Web Services. Documentación de Amazon S3.
- Amazon Web Services. Documentación de Amazon CloudWatch.
- Amazon Web Services. Documentación de AWS IAM.
- Flask Documentation.
- Gunicorn Documentation.
- Nginx Documentation.

A. Evidencias de implementación

Las evidencias completas de configuración y funcionamiento se encuentran almacenadas dentro del repositorio `github.com/paul-pinto/reto20-portal-academico` en la carpeta docs/evidencias. En el cuerpo principal se incluyeron únicamente las capturas necesarias para demostrar arquitectura, persistencia, almacenamiento privado, autorización, observabilidad y control de costos.

Cuando corresponde, las capturas conservan visible la región de AWS y la fecha o contexto temporal suficiente para demostrar que pertenecen a la ejecución del proyecto.

Como respaldo adicional, el conjunto de evidencias incluye: detalles de EC2 y Security Groups; registros y entidades de DynamoDB; configuración de cifrado, versionado y objetos de S3; pantallas de autenticación y navegación por rol; casos permitidos y denegados; eventos de CloudWatch; estado del CloudWatch Agent, Gunicorn y Nginx; créditos del Learner Lab; y estado final de los recursos.